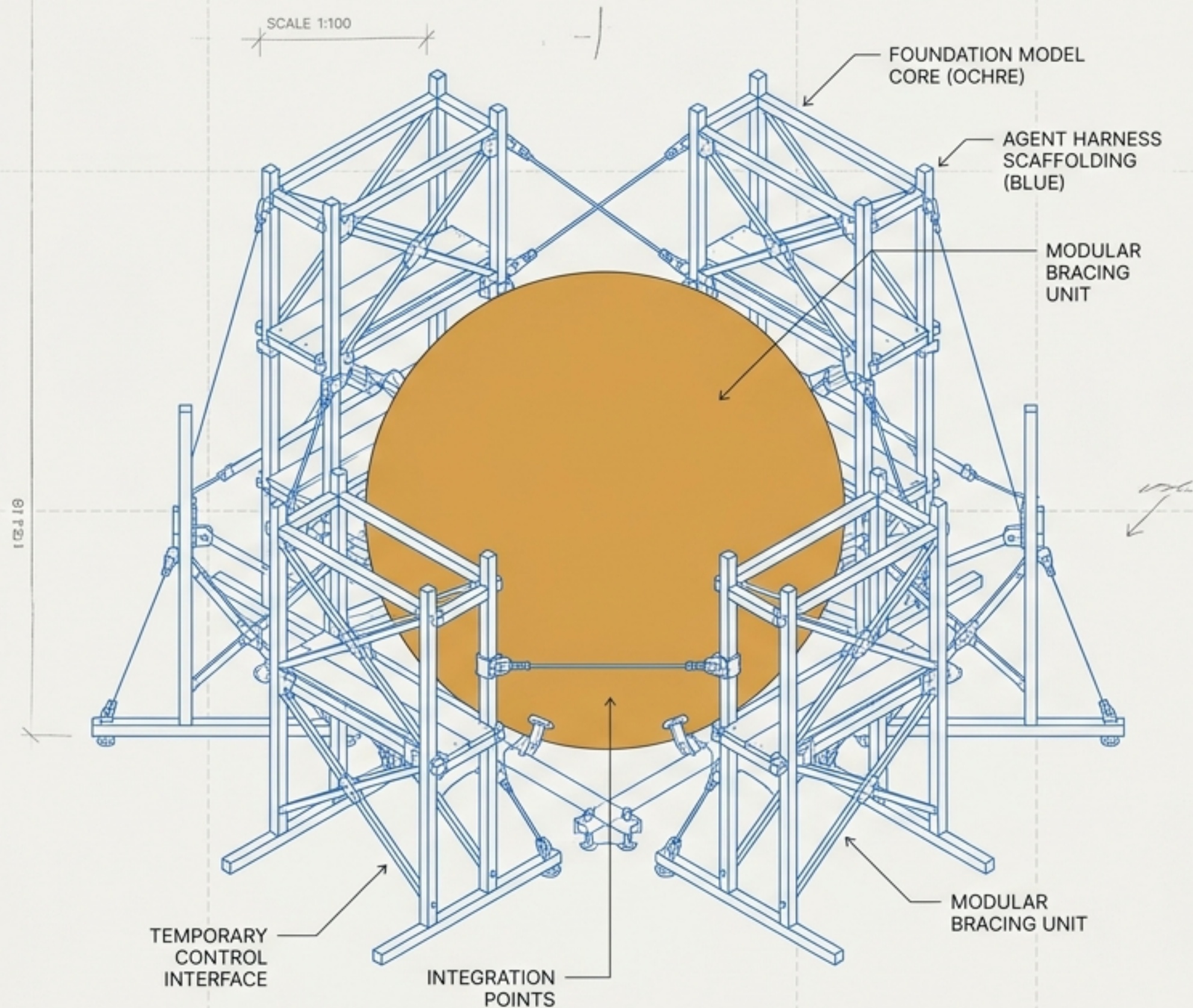


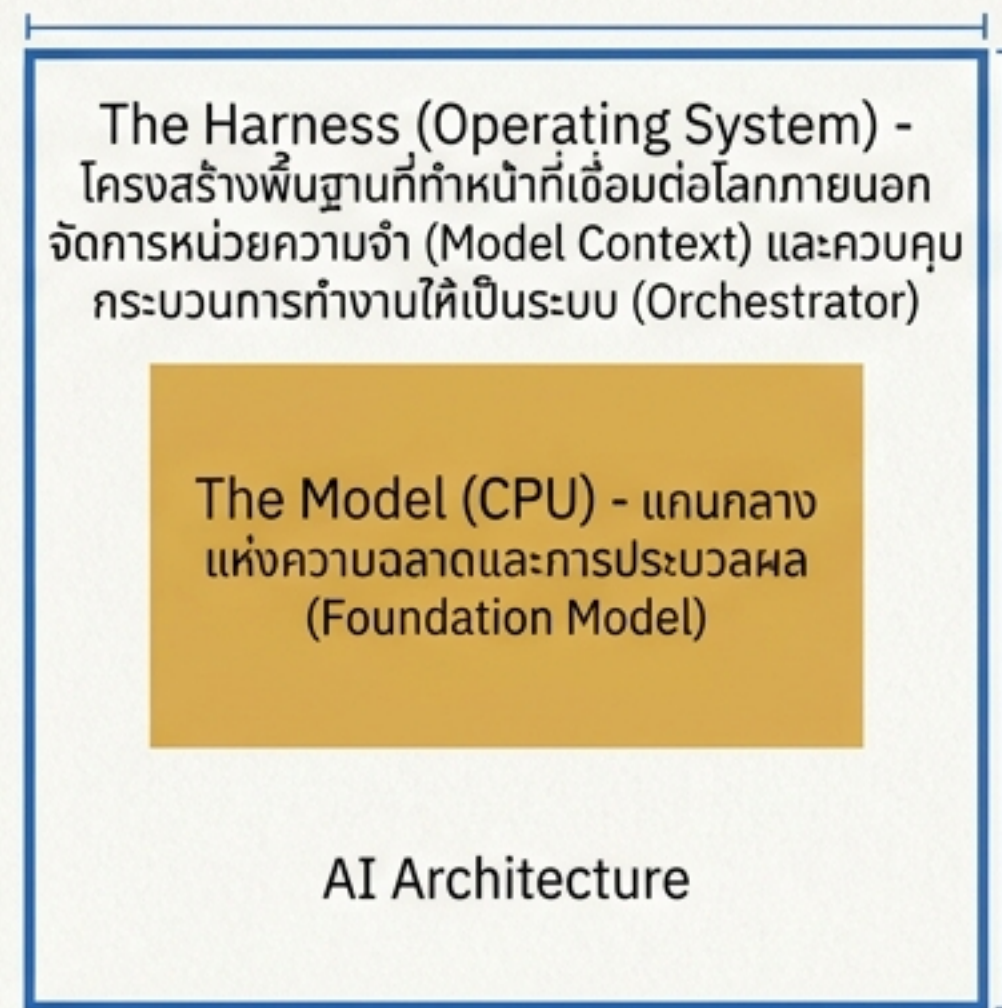
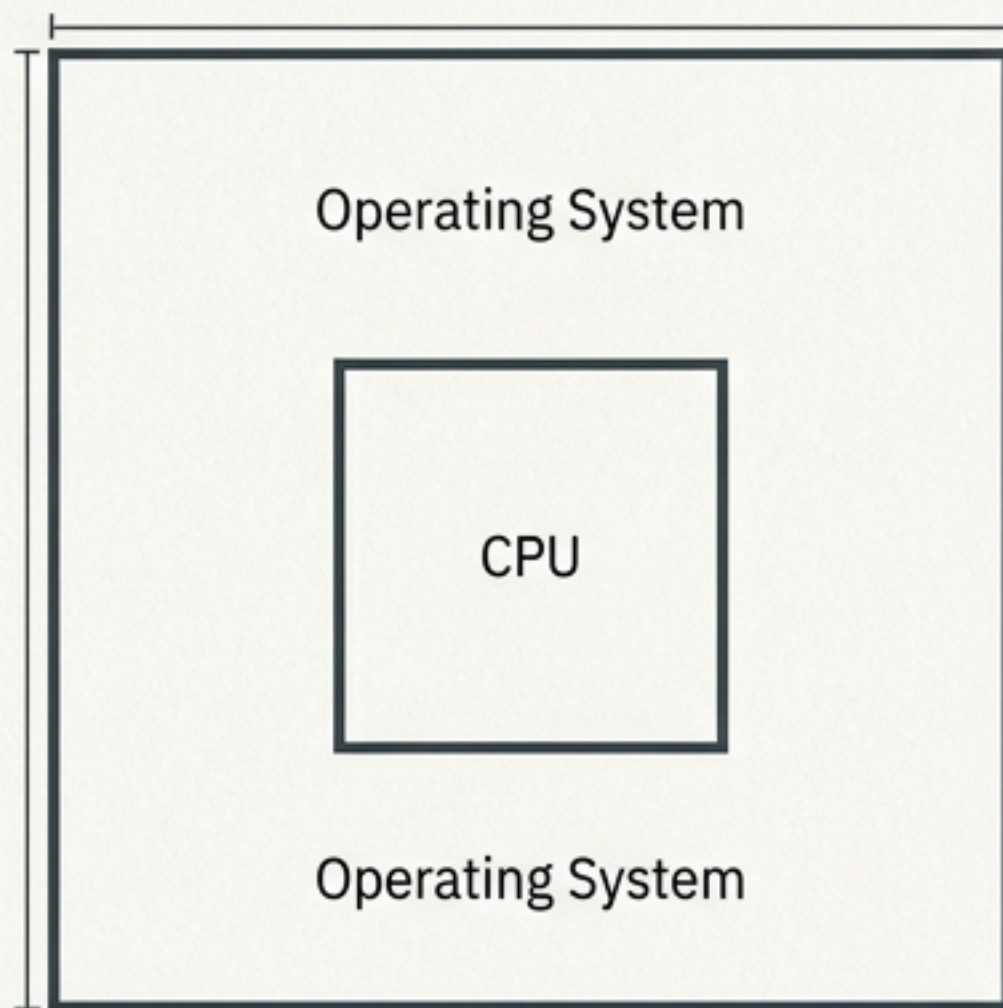


หน้าทางเทคนิคที่ซ่อนอยู่ ของระบบ AI: สถาปัตยกรรม Agent Harness

การออกแบบโครงสร้างควบคุมที่พร้อม
ถูกทำลายและกลืนกินโดย AI รุ่นถัดไป

ปัญญาประดิษฐ์ในฐานะระบบคอมพิวเตอร์ยุคใหม่

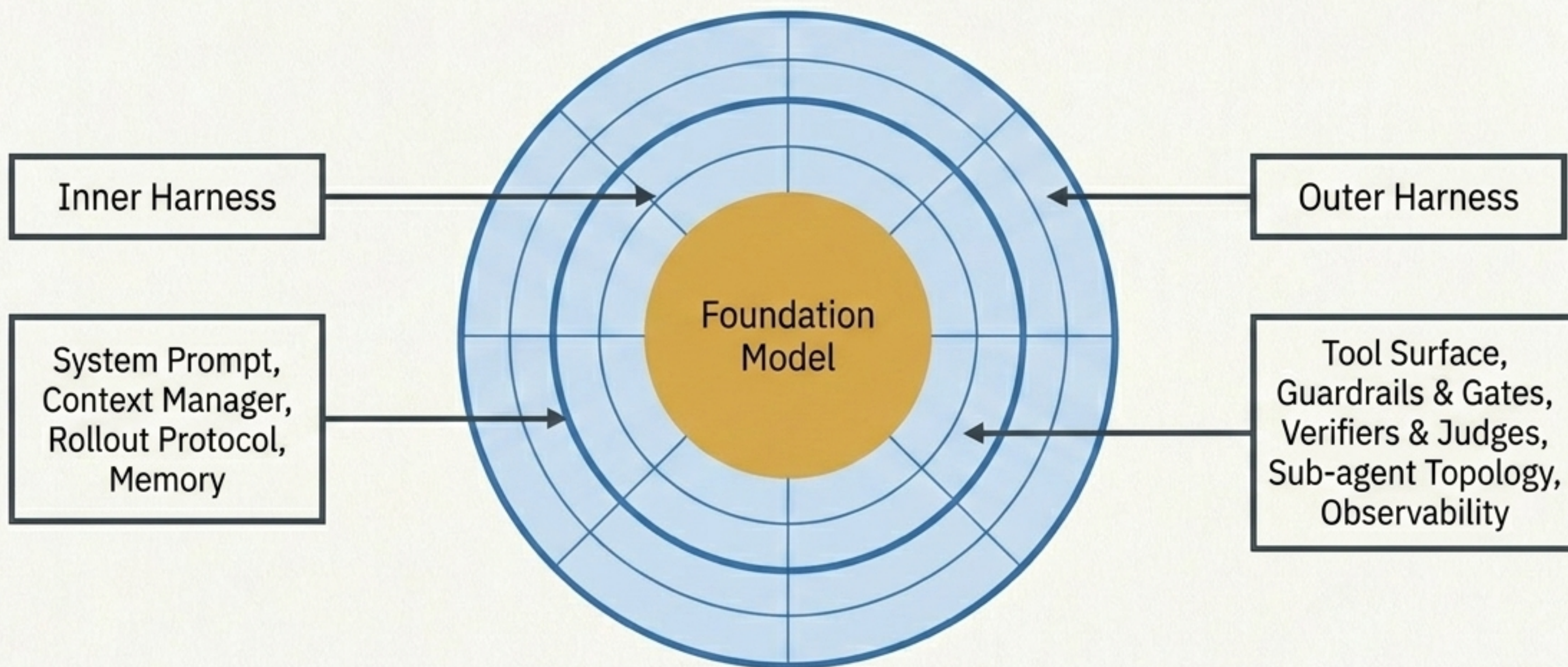
โมเดล AI เพียงอย่างเดียวไม่สามารถทำงานได้หากปราศจากระบบปฏิบัติการที่คอยจัดการคำสั่งและทรัพยากร



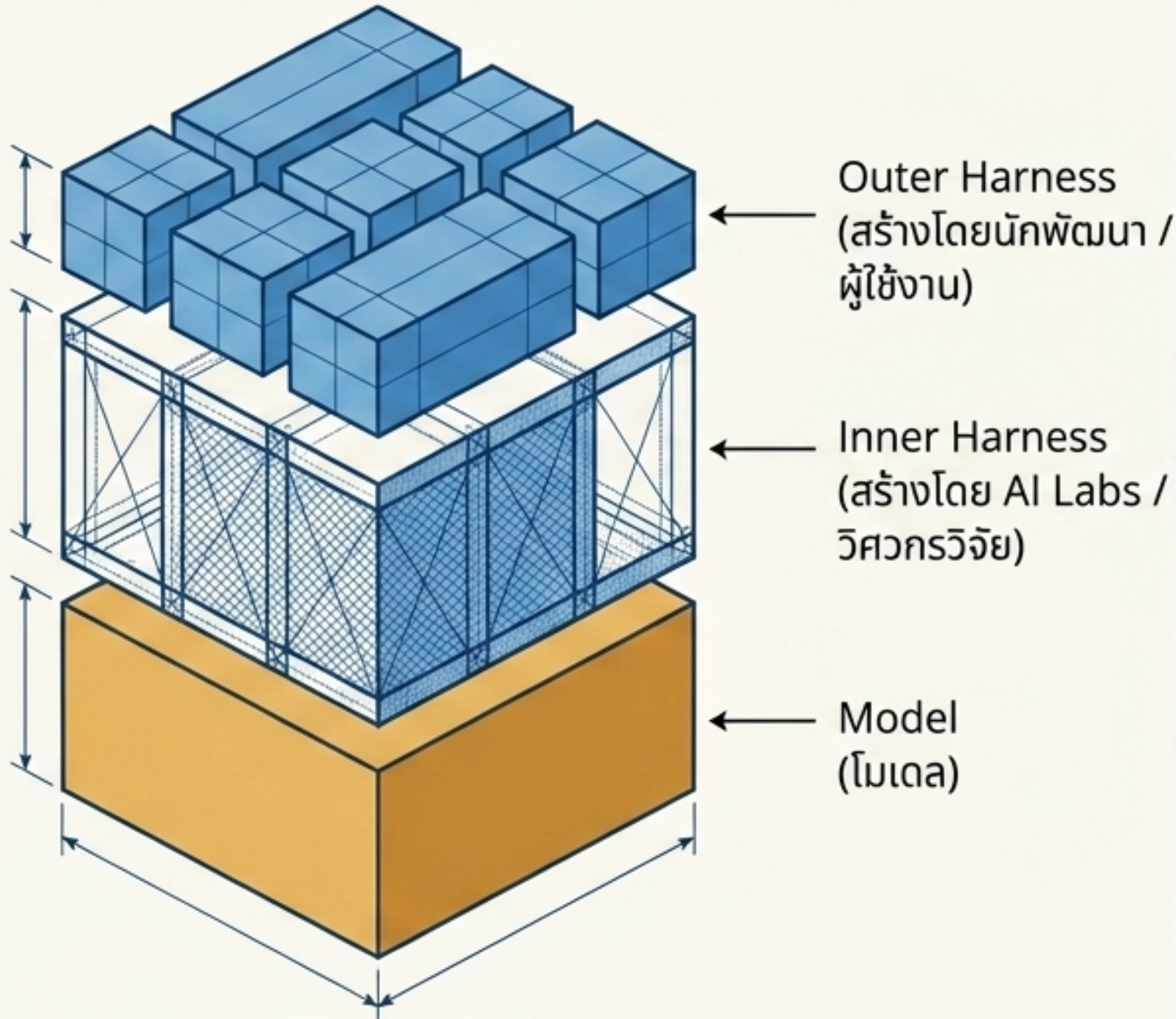
Agent ที่สมบูรณ์แบบ = โมเดลพื้นฐาน + โครงสร้าง Harness

สถาปัตยกรรม 9 องค์ประกอบของระบบ Agent

Harness ไม่ใช่แค่เชื่อมต่อ (Plumbing) แต่เป็นวงจรป้อนกลับ (Feedback Loop) ที่ถูกออกแบบมาโดยเฉพาะเพื่อแปลงการเรียกใช้โมเดลหนึ่งครั้งให้กลายเป็นผลลัพธ์ที่ใช้งานได้จริง



สถาปัตยกรรมภายในและภายนอก (Inner vs. Outer Harness)

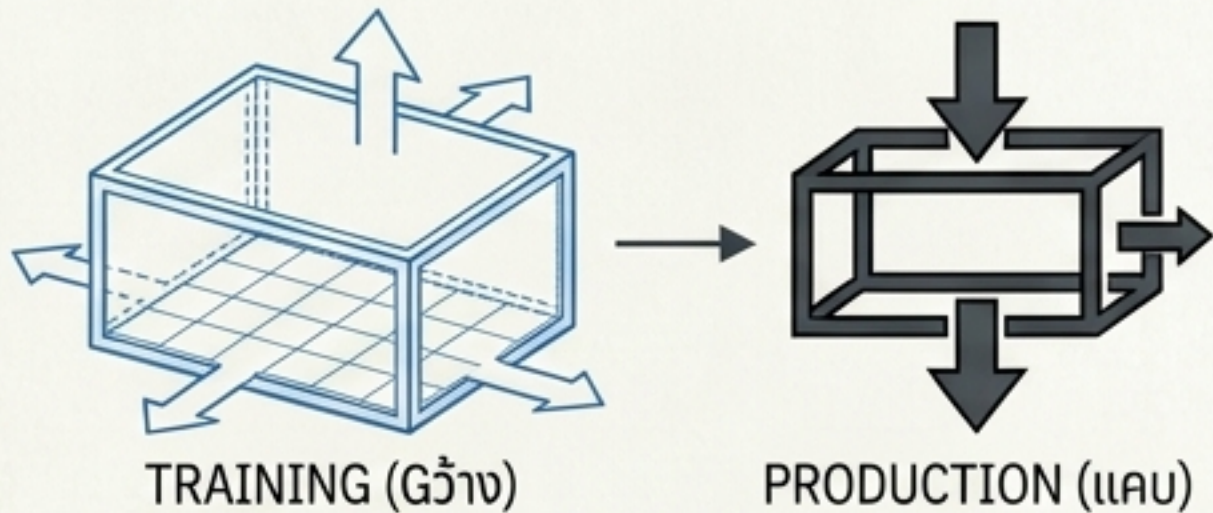


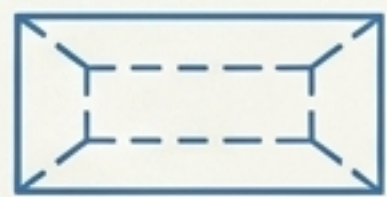
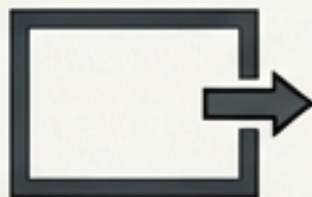
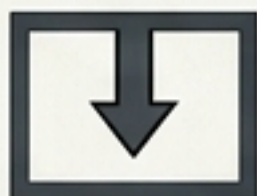
โครงสร้างแต่ละชั้นวิวัฒนาการด้วยความเร็วที่ต่างกัน และสะสม 'หนี้ทางเทคนิค' ในรูปแบบที่ต่างกัน:

- Inner Harness: โครงสร้างระดับลึกที่ควบคุมการทำงานพื้นฐานของโมเดล
- Outer Harness: การประกอบทักษะเฉพาะทาง เครื่องมือภายนอก และกฎระเบียบขององค์กร กับลงบนโมเดล

ความไม่สมมาตรที่ยิ่งใหญ่: สภาพแวดล้อมการฝึกสอน vs. การใช้งานจริง

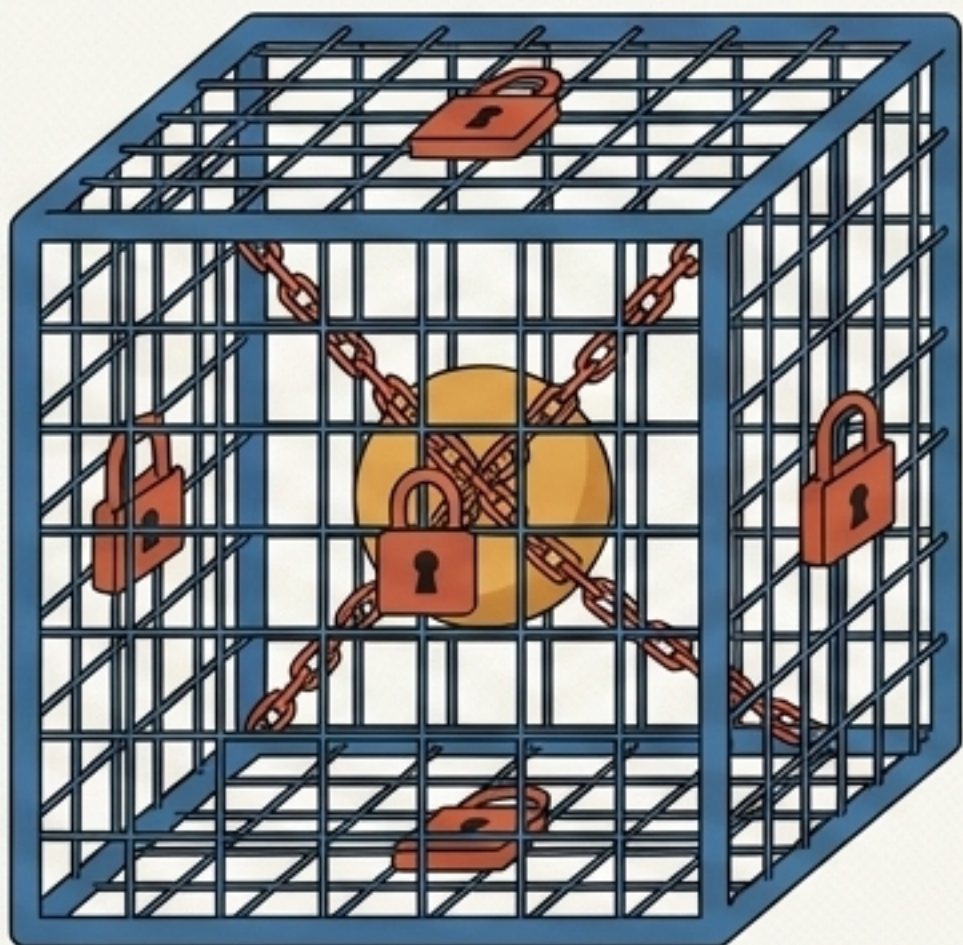
Harness สำหรับ Training ต้อง ‘กว้าง’ เพื่อการสำรวจ ในขณะที่
Harness สำหรับ Production ต้อง ‘แคบ’ เพื่อควบคุมข้อจำกัด



มิติการทำงาน (Dimension)		Training / Research Harness (พื้นที่สำรวจ)	 Production Harness (พื้นที่ควบคุม)
พื้นที่การกระทำ (Action Space)		กว้างที่สุด: ให้โมเดลทดลอง ทำทุกอย่างที่เป็นไปได้	 แคบที่สุด: อนุญาตเฉพาะสิ่งที่ กำหนด (Deny by default)
เครื่องมือ (Tools)		ระดับล่างแบบดิบ (Raw), ขยายผลได้, โมเดลเขียนเองได้	 ถูกครอบไว้ (Wrapped), จำกัด ขอบเขต, ตรวจสอบ Schema
ความล้มเหลว (Failures)		ยอมรับได้: เป็นสัญญาณ (Signal) สำหรับ Optimizer	 ต้องถูกระงับ: Fail closed, ระบบ Retry, แจ้งเตือนมนุษย์
สถานะ (State)		สามารถย้อนกลับได้ (Replayable / Snapshottable)	 บันทึกถาวร, แยกตามผู้ใช้, ตรวจสอบย้อนหลังได้ (Auditable)

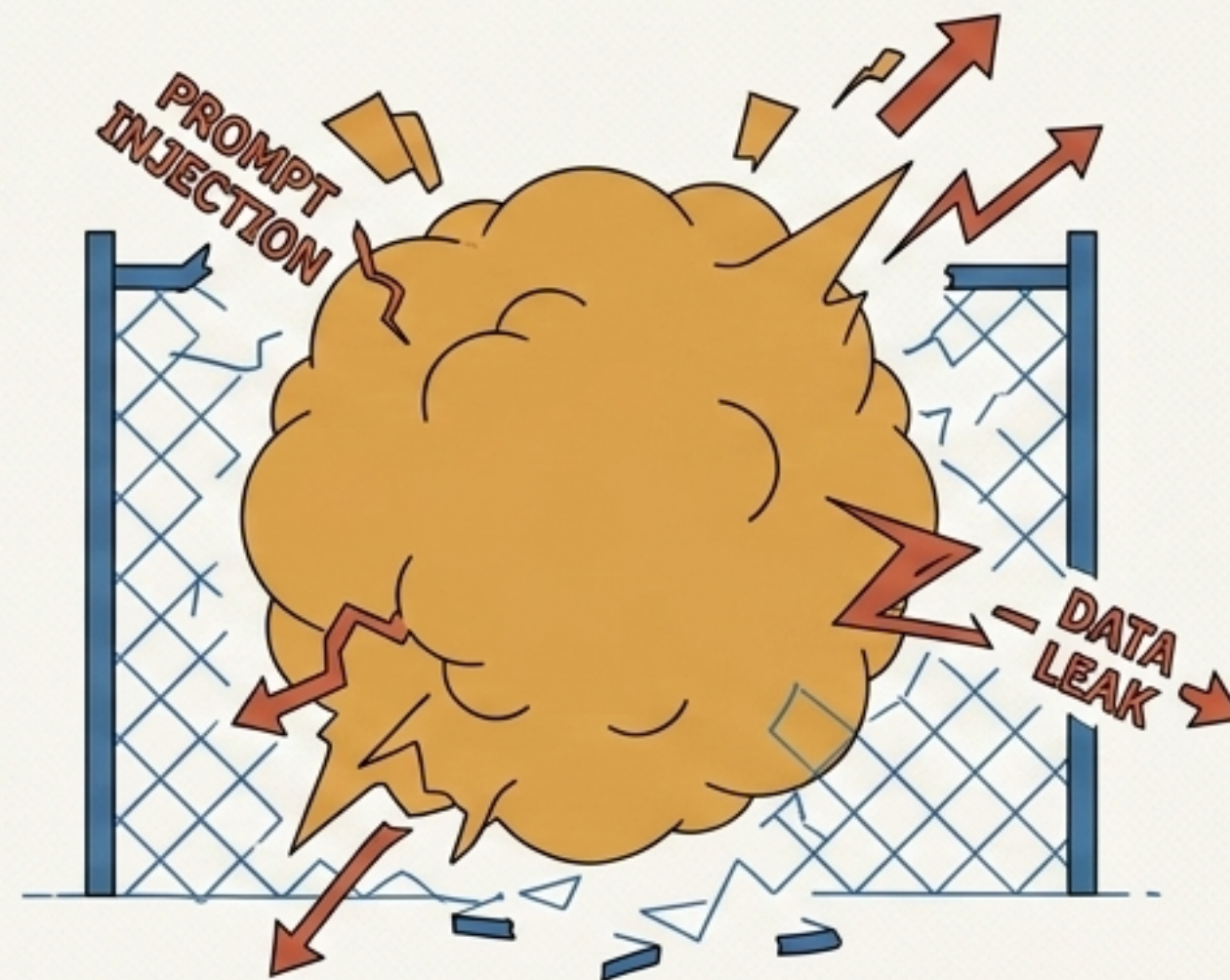
ความล้มเหลว 2 รูปแบบของการออกแบบสถาปัตยกรรม

1. การตีกรอบแน่นเกินไปตอนฝึก (Over-shackling in Training)



นำข้อจำกัดของ Production มาใช้ตอน Training ส่งผลให้โมเดลไม่เคยเรียนรู้ที่จะกู้คืนจากข้อผิดพลาด หรือเลือกเครื่องมือด้วยตัวเอง (ซอฟต์แวร์คิดแทนโมเดล)

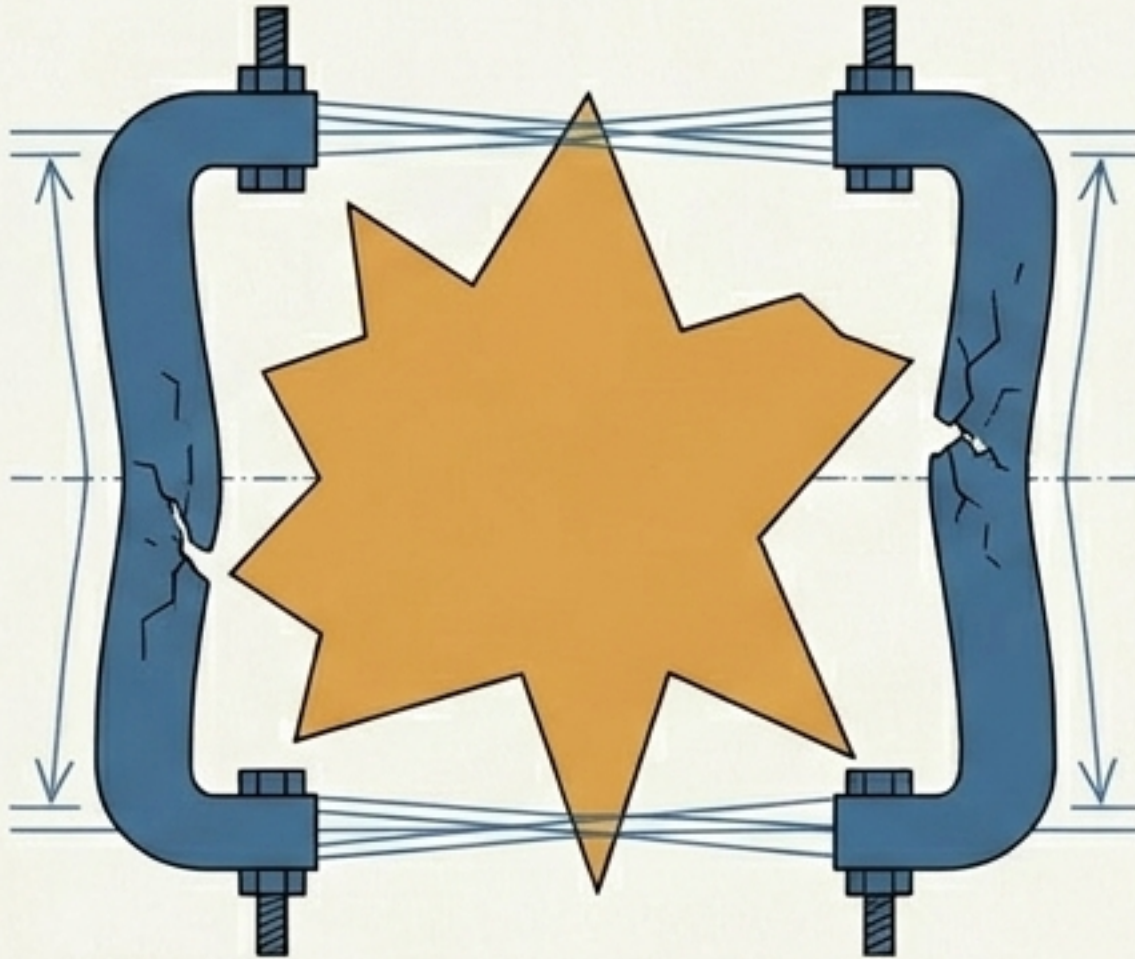
2. การป้องกันหละหลวมตอนใช้งานจริง (Under-fencing in Production)



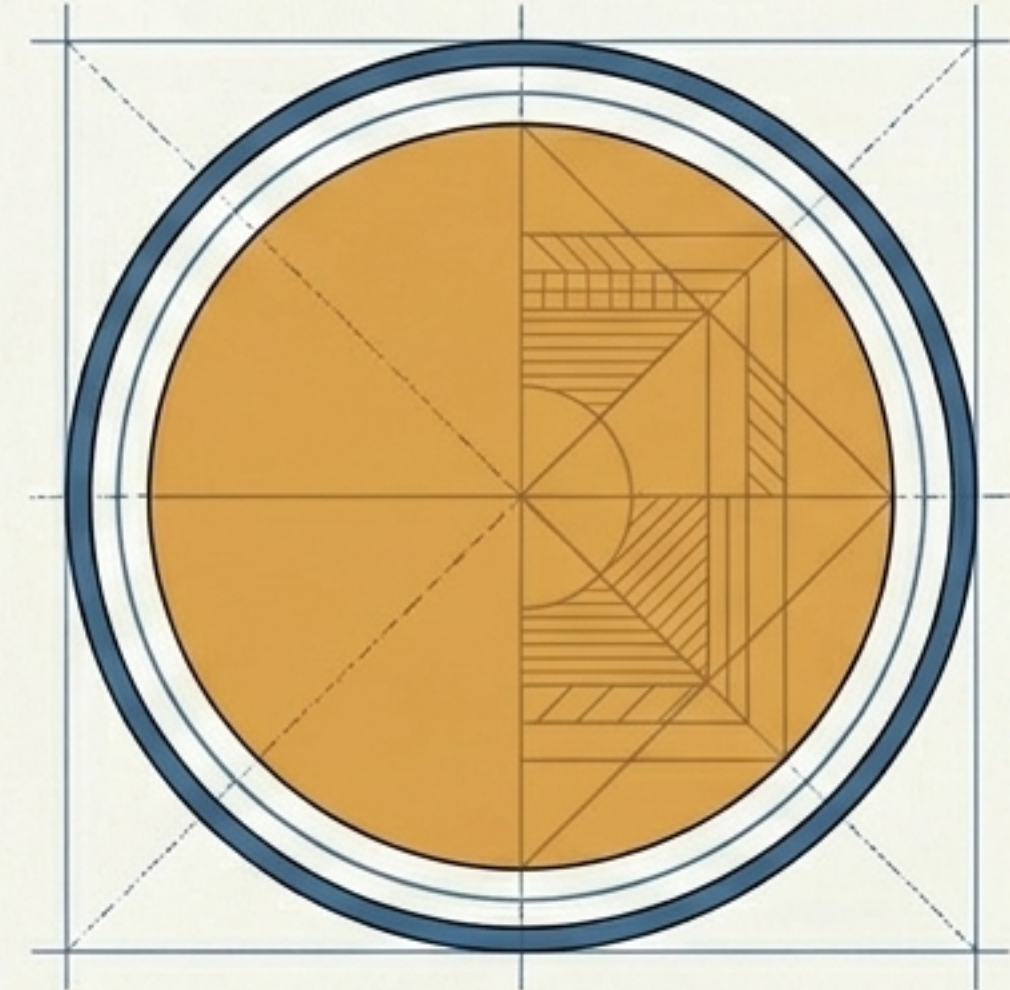
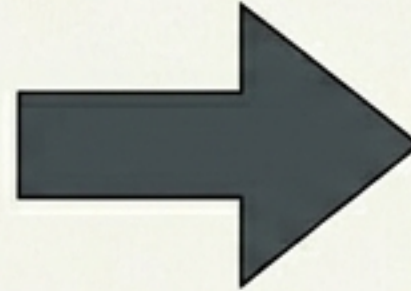
นำโมเดลที่ฝึกมาแบบเปิดกว้างไปใช้งานจริงโดยไม่มีระบบควบคุม นำไปสู่ช่องโหว่ (Prompt Injection) และการรั่วไหลของข้อมูล

การจัดศูนย์ (Alignment): จากภายในสู่ภายนอก

การเพิ่มตรรกะซอฟต์แวร์รอบๆ โมเดล ไม่ใช่วิธีการแก้ปัญหายที่ยั่งยืน



วิศวกรรมซอฟต์แวร์ (ภายนอก): การสร้างรั้วกัน (Allowlists, Filters) ซึ่งมีข้อจำกัดและจะถูกทำลายได้เมื่อโมเดลฉลาดขึ้น



วิศวกรรมสภาพแวดล้อม (ภายใน): การให้โมเดลสำรวจและเรียนรู้ผ่าน Reward Shaping รั้วกันจะถูกย้ายจากภายนอกเข้าไปเป็นนโยบายภายในของโมเดลเอง (Internal Policy)

“โมเดลที่มีนโยบายภายในที่ดี ต้องการโครงสร้างภายนอกน้อยลง”

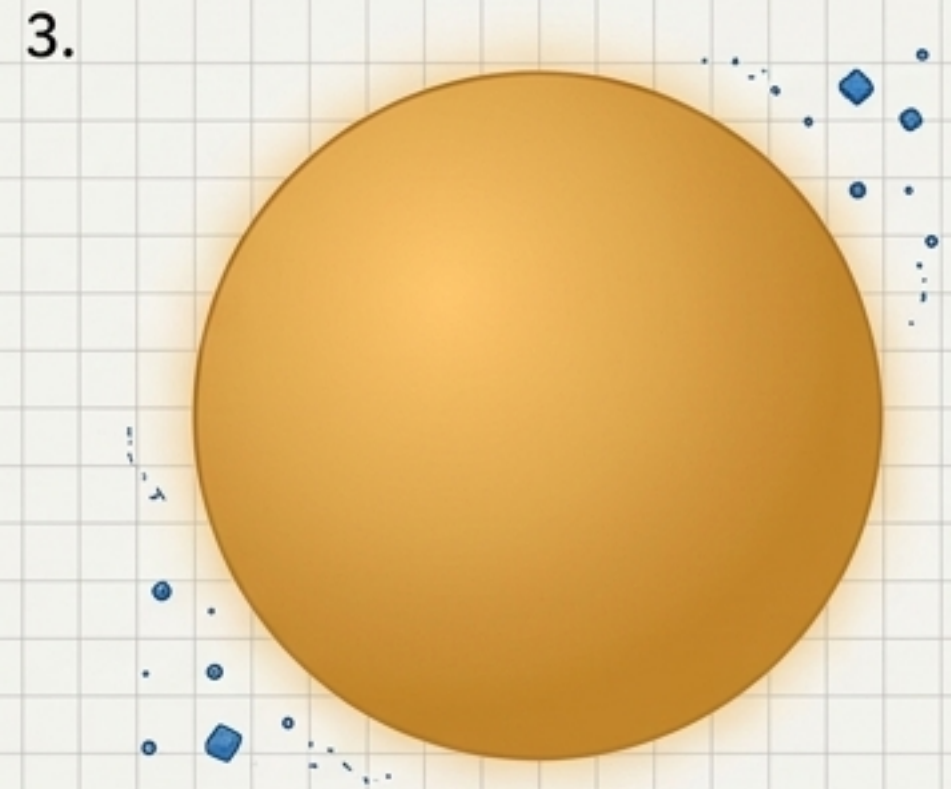
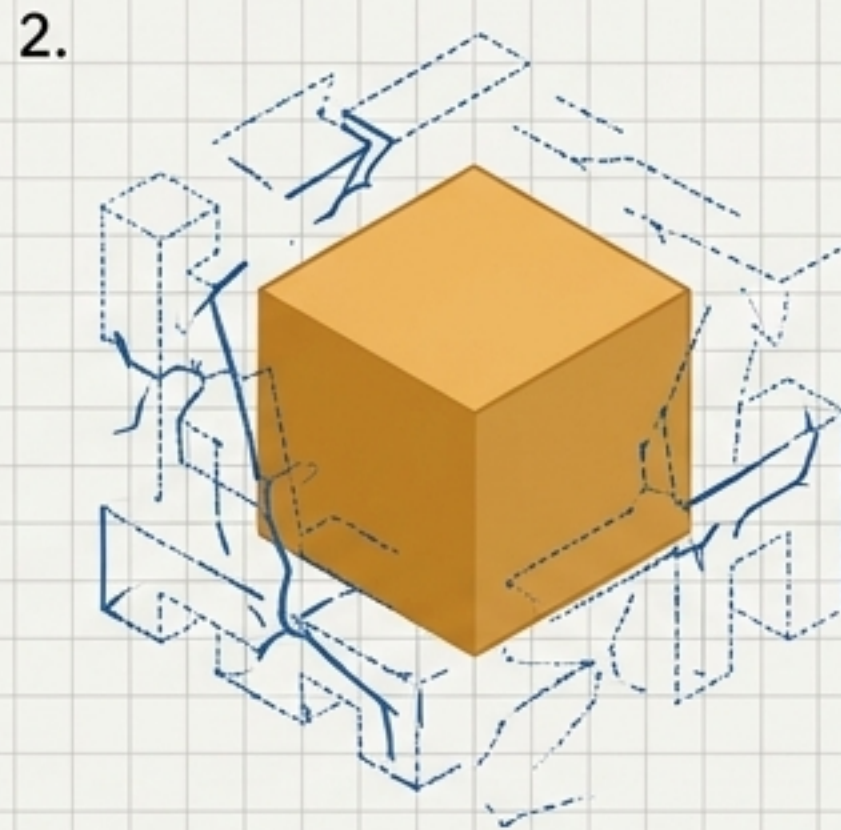
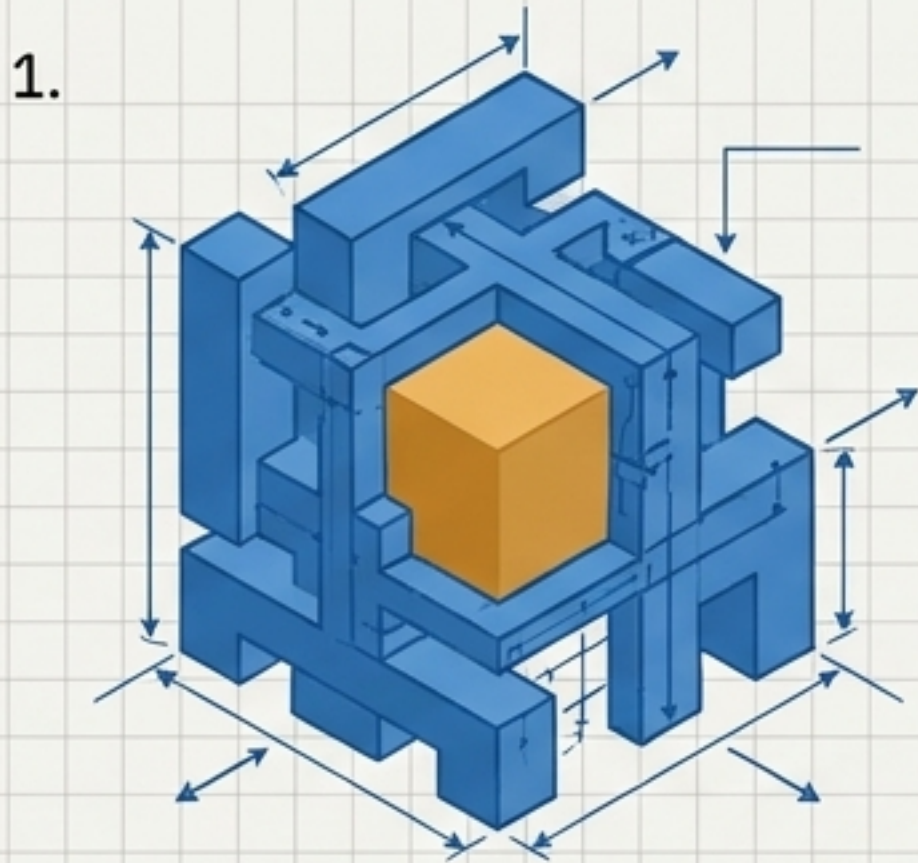
First-Party vs. Third-Party Harness: ช่องว่างของประสิทธิภาพ



- โดยปกติ First-Party Harness (เช่น Claude Code บนโมเดล Claude) จะชนะเสมอ เพราะโมเดลถูกฝึกมาให้ตอบสนองกับโครงสร้างของตัวเองโดยเฉพาะ
- ข้อยกเว้น: Third-Party สามารถชนะได้ หากลงทุนมหาศาลในแกนความสามารถที่ First-Party มองข้าม
- กรณีศึกษา: Letta Code เอาชนะ Claude Code บน Opus 4.5 ได้อย่างขาดลอยใน Benchmark ด้านหน่วยความจำ เนื่องจากโครงสร้าง Letta ออกแบบมาเพื่อเน้น Durable Memory เป็นหลัก

บทเรียนอันขมขื่น (The Bitter Lesson)

โครงสร้าง Harness ทั้งหมดคือสิ่งที่คุณจำเป็นต้องมี “สำหรับระดับความสามารถของโมเดลในปัจจุบันเท่านั้น”

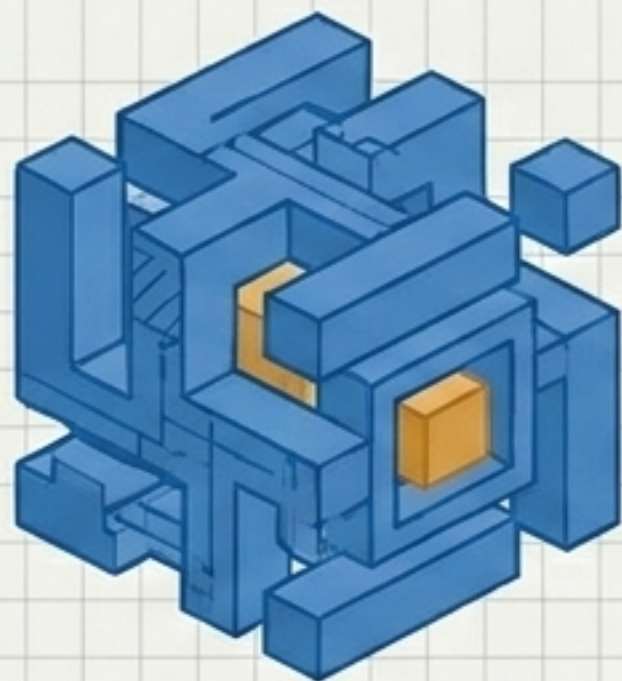


“จงสร้างโครงสร้างพื้นฐานสำหรับระดับของพลังประมวลผลที่คุณมีในวันนี้ จากนั้นจงรี้อถอนมันทิ้ง เพราะโครงสร้างนั้นจะกลายเป็นคอกขวิดสำหรับพลังประมวลผลในยุคถัดไป” – Hyung Won Chung

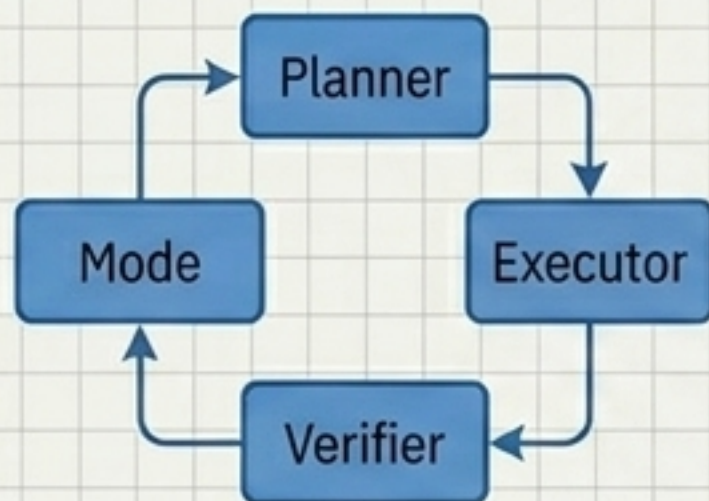
วิศวกรรมที่ใช้แก้ไขข้อจำกัดของโมเดล จะถูกกลืนกินด้วยโมเดลรุ่นต่อไปที่มีความสามารถสูงขึ้นเสมอ

โทม์ไลน์การเสื่อมสลายของโครงสร้าง (Structure Erosion)

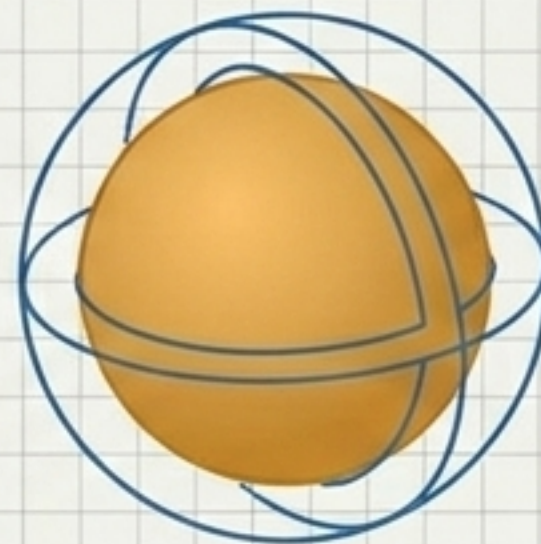
วงจรชีวิตของความซับซ้อนทางวิศวกรรมที่สั้นลงเรื่อยๆ



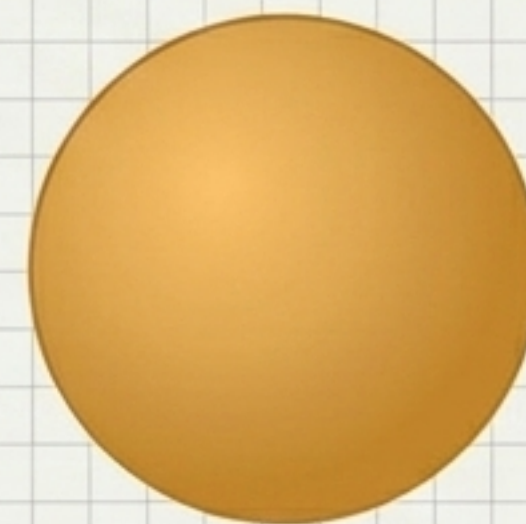
2023



2024



2025



2026

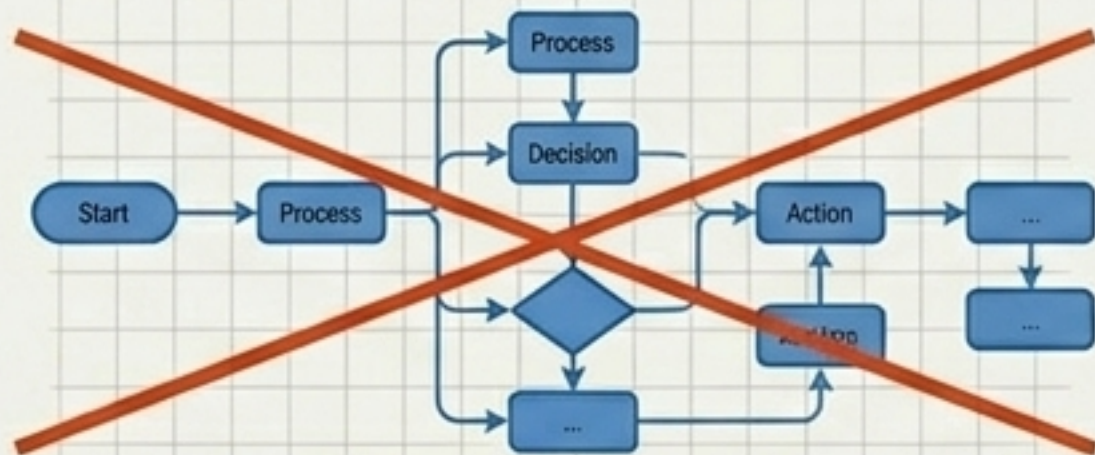
ยุคของ RAG Pipeline
— Harness ต้องรับภาระ
ด้านหน่วยความจำทั้งหมด
เพราะโมเดล Context สั้น

ยุคของ Workflows
— ต้องสร้างระบบ
Planner-Executor แยกส่วน
เพราะโมเดลใช้เครื่องมือ
แบบวนลูปไม่แม่นยำ

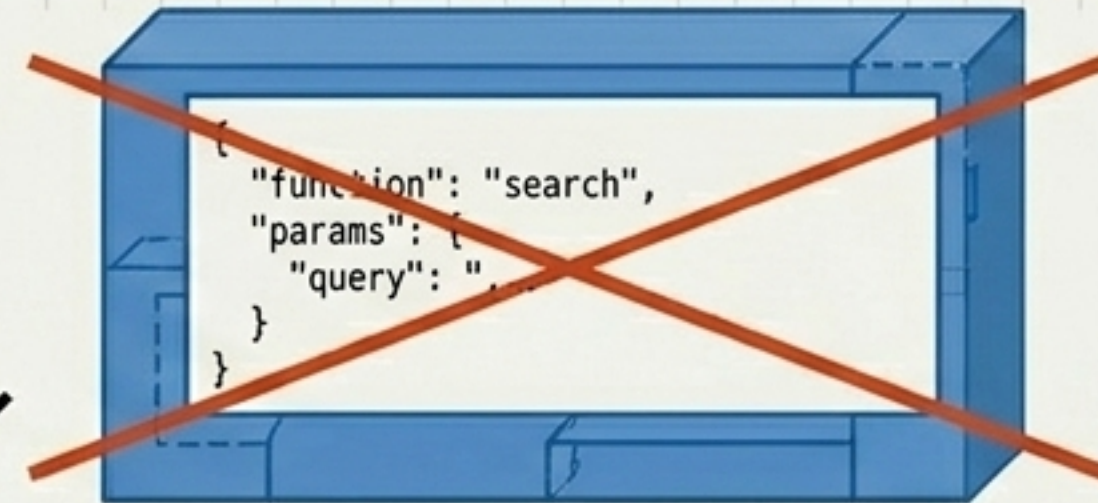
ยุคของการใช้เครื่องมือ
— MCP เริ่มแพร่หลาย
โมเดลเริ่มใช้งาน Tool
Calling และ Reasoning
แบบฝังตัว

ยุคของ Agentic Thinking
— โมเดลจัดการการวางแผน
ทำงาน และตรวจสอบได้
ด้วยตัวเองเบ็ดเสร็จ

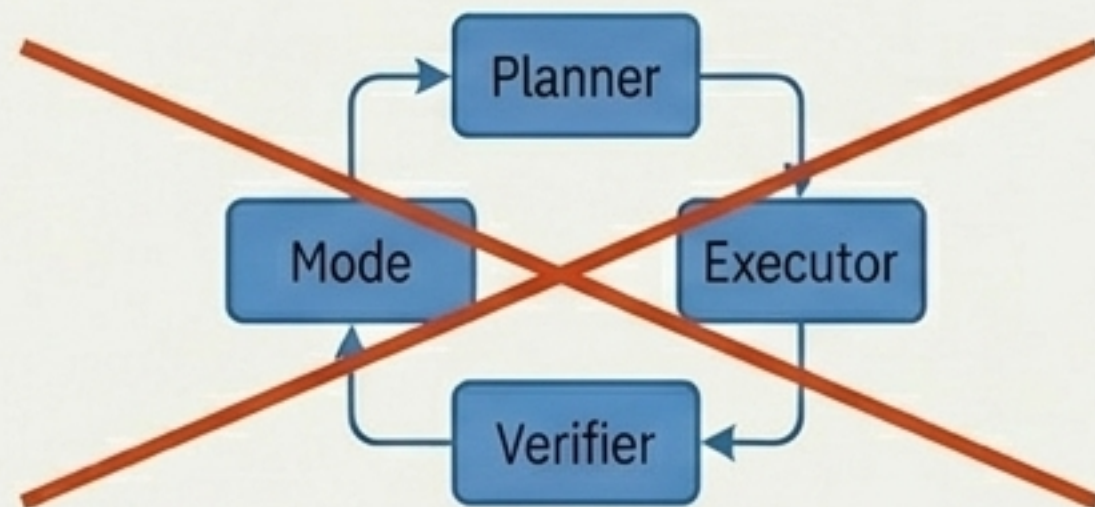
4 โครงสร้างที่กำลังจะเลือนหายไปโมเดลยุคหน้า



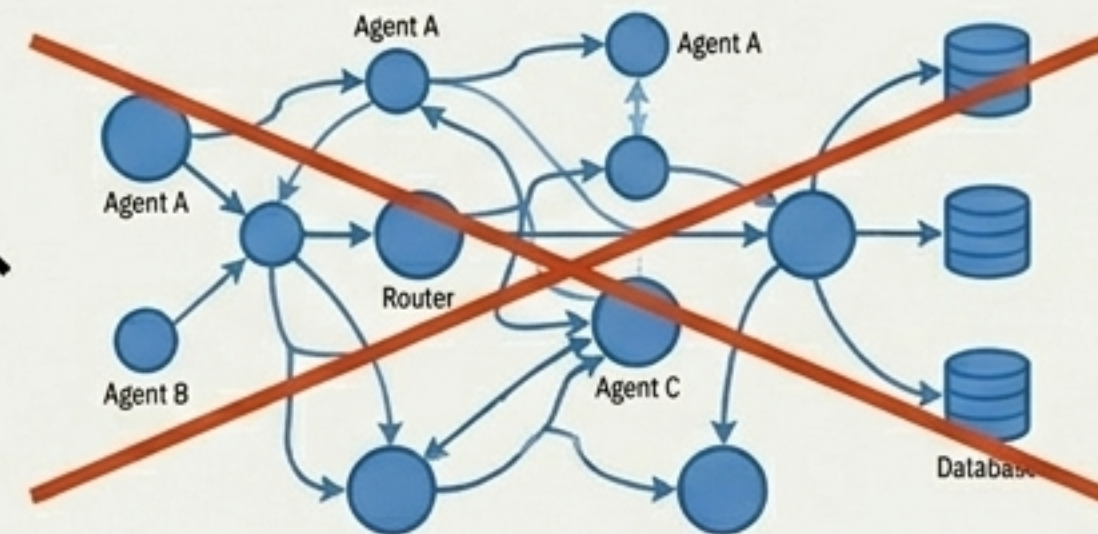
1. No-Code Workflow Builders: การลากเส้นเชื่อมต่อโหนด จะถูกแทนที่ด้วย Agent ตัวเดียวที่คิดแบบ Long-horizon



2. Tool Wrappers: การครอบ API ให้ย่อยง่ายจะไม่จำเป็น เมื่อโมเดลสามารถอ่าน OpenAPI Specs และ DOM ได้โดยตรง



3. Planner-Executor Scaffolds: โค้ด Python ที่ใช้สั่งการและตรวจสอบ จะถูกยุบรวมเป็น Trace เดียวในความคิดของโมเดล



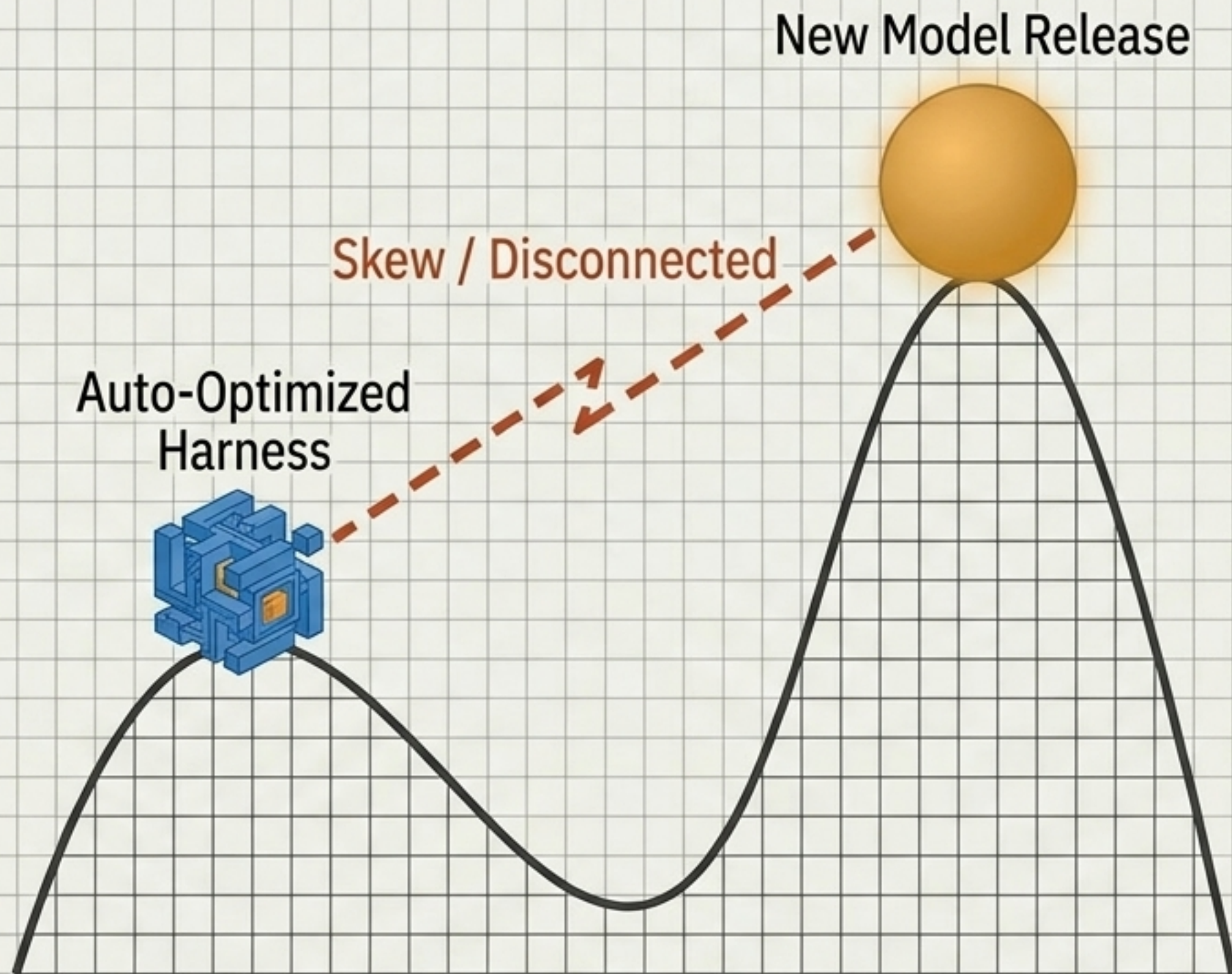
4. Multi-Agent Topologies: สถาปัตยกรรมเครือข่ายเอเจนต์ที่ซับซ้อน จะถูกลดรูปเหลือเพียงระบบ Context Isolation และ Plain text

Agentic Model

หลุมพรางของการทำ Auto-Optimization

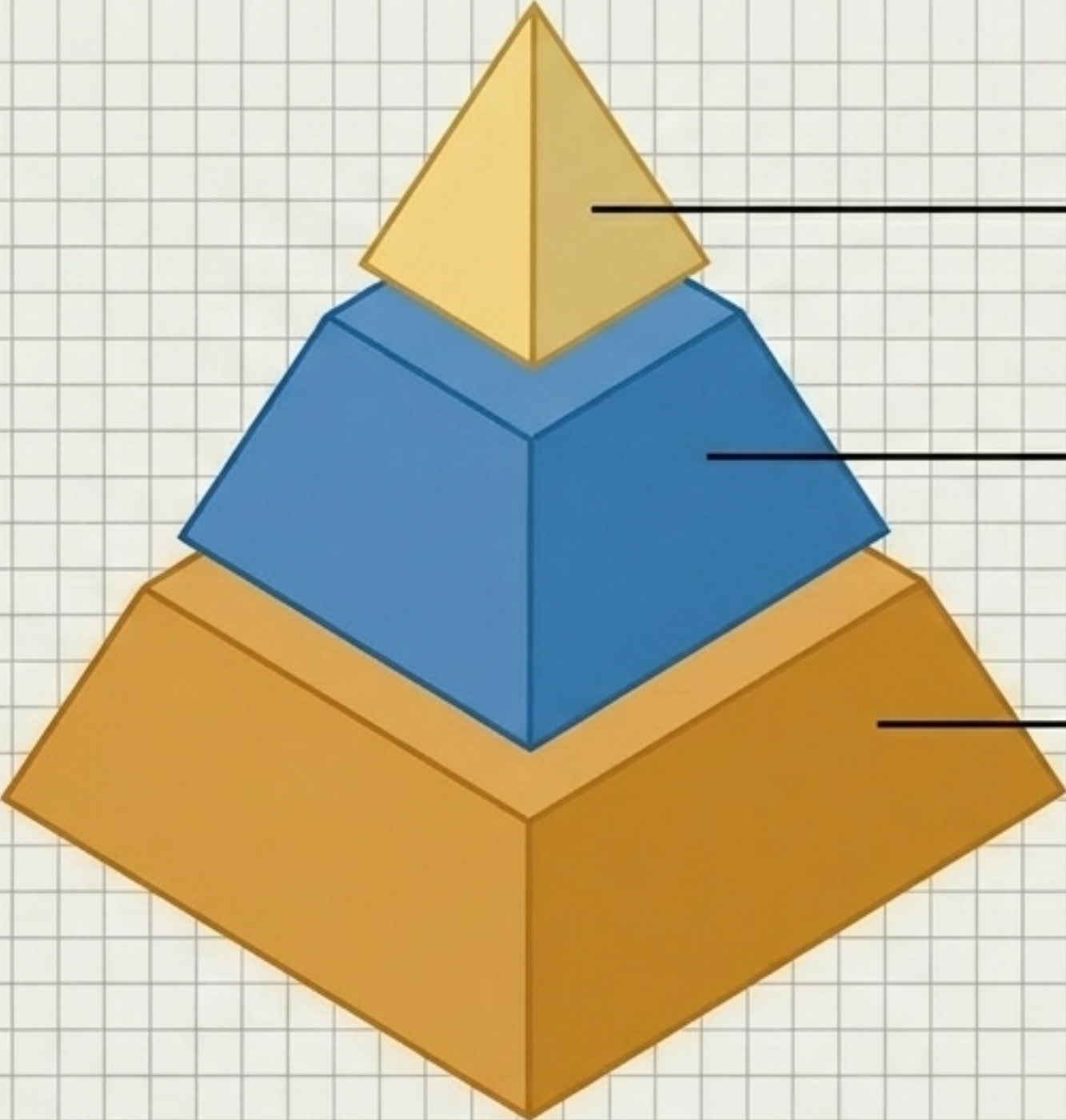
การใช้อัลกอริทึมปรับจูน Harness อัตโนมัติ (เช่น Meta-Harness) ได้ผลดีในระยะสั้น แต่สร้างหิ้งมหาศาลในระยะยาว

- **Overfit กับงานปัจจุบัน:** โครงสร้างเรียนรู้ที่จะจดจำจุดอ่อนของโมเดลปัจจุบัน เมื่อเปลี่ยนโมเดล โครงสร้างนี้จะพังทันที
- **ขยายช่องว่าง Train/Prod Skew:** ห้องปฏิบัติการ (Lab) ฝึกโมเดลใน Harness แบบหนึ่ง แต่คุณรัน Auto-optimizer เพื่อสร้าง Harness อีกแบบหนึ่ง ทำให้เกิดความคลาดเคลื่อนที่ไม่สามารถตรวจสอบด้วยมนุษย์ได้ (Opacity)

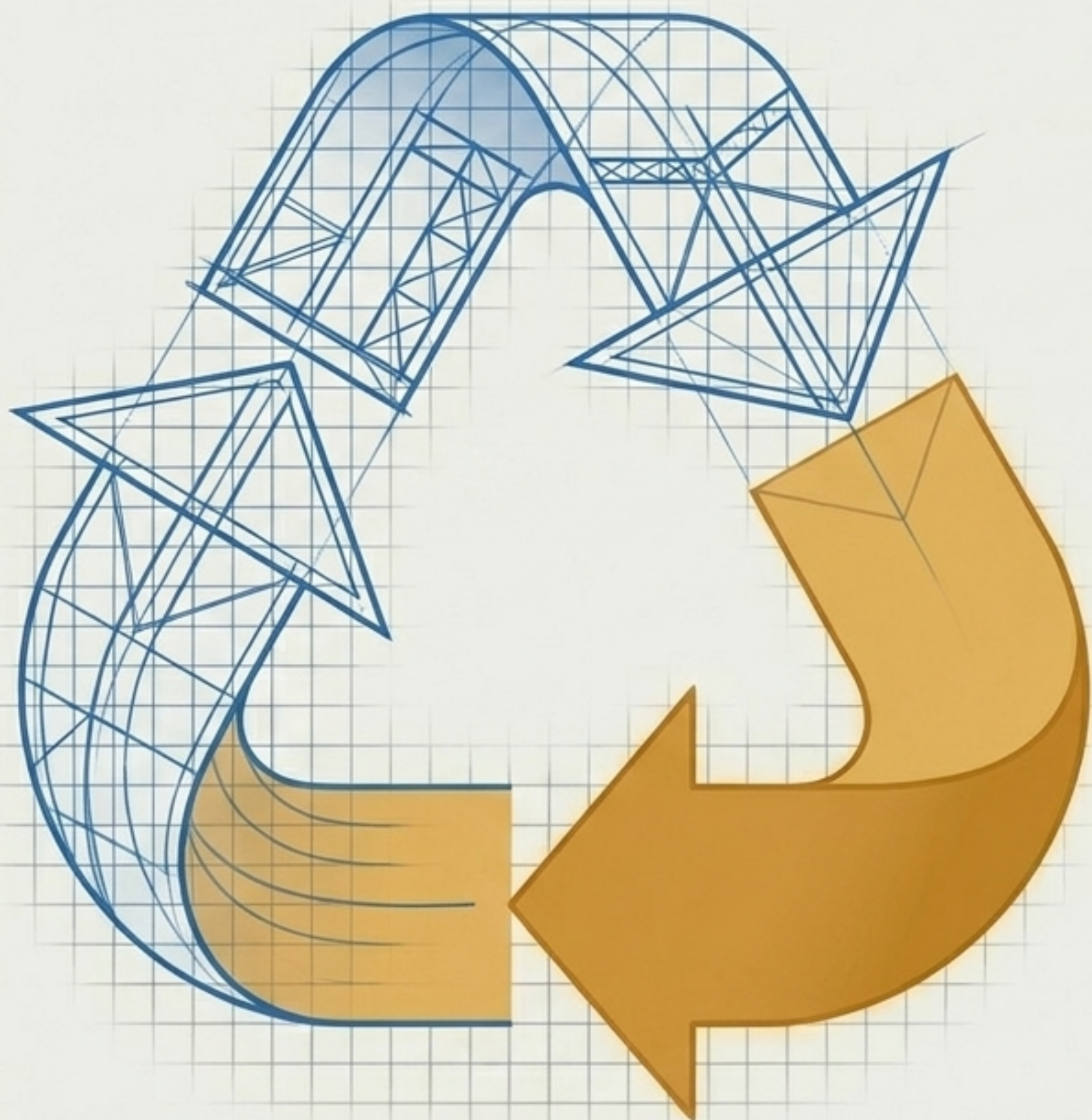


พื้นผิวการปรับแต่ง 3 ระดับ (Three Optimization Surfaces)

จัดสรรงบประมาณและเวลาของคุณให้อยู่ในชั้นที่คุ้มค่าที่สุด: ‘ผลกระทบงานไปยังชั้นที่ต้นทุนต่ำที่สุด’



ชั้น (Surface)	ต้นทุนการปรับเปลี่ยน	ความถี่ในการอัปเดต	ผู้รับผิดชอบ
Skills / Prompts	ถูกที่สุด (แก้ไขข้อความได้ทันที)	ทุกชั่วโมง / ทุกวัน	Product Builders
Agent Harness	ปานกลาง (ต้องแก้ไขโค้ดและ Deploy ใหม่)	ทุกวัน / ทุกสัปดาห์	Applied AI Engineers
Foundation Model	แพงที่สุด (ใช้พลังประมวลผลมหาศาล)	ทุกไตรมาส	Research Labs



วงจรรีไซเคิล 90 วัน: ‘Thin Harness, Fat Skills’ Harness บางเบา, ทักษะจัดเต็ม

จงออกแบบโครงสร้างฝังแอปพลิเคชันให้เป็น
‘วัตถุชั่วคราวที่มีอายุ 90 วัน’ ไม่ใช่สถาปัตยกรรมถาวรขององค์กร

ทีมที่จะอยู่รอดเมื่อโมเดลรุ่นใหม่มารถึง คือทีมที่:

1. จัดการ Harness ให้บางที่สุด โดยยึดตามรูปแบบที่ Lab ใช้เทรนโมเดล
2. ย้ายความเชี่ยวชาญเฉพาะทางไปไว้อยู่ที่ชั้น Skills ซึ่งปรับแก้ได้ง่าย
3. วางโครงสร้างโค้ดให้พร้อมที่จะ ‘ลบทิ้งโดยไม่เสียดาย’ ในทันทีที่มีการเปิดตัวโมเดลรุ่นถัดไป